

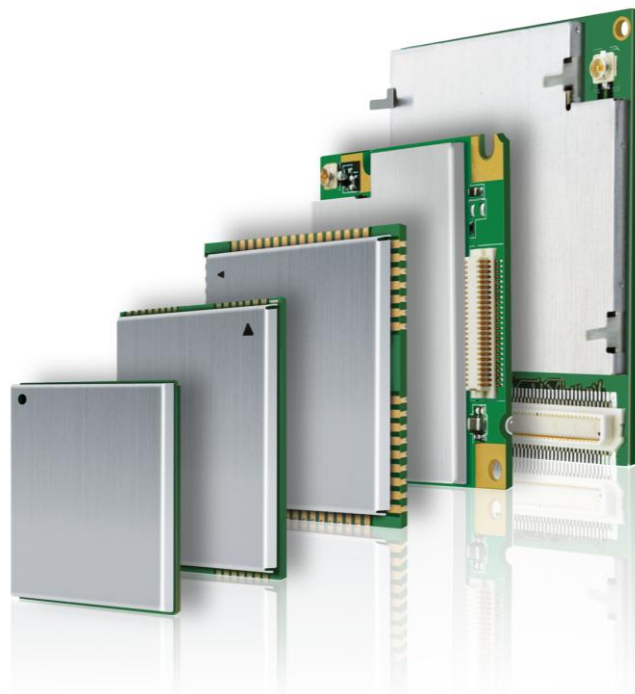


OpenCPU

Quectel Cellular Engine

OpenCPU FOTA Application Notes

OPEN_CPU_FOTA_AN_V1.0



Document Title	OpenCPU FOTA Application Notes
Version	1.0
Date	2012-09-11
Status	Released
Document Control ID	OPEN_CPU_FOTA_AN_V1.0

General Notes

Quectel offers this information as a service to its customers, to support application and engineering efforts that use the products designed by Quectel. The information provided is based upon requirements specifically provided for Quectel by the customers. Quectel has not undertaken any independent search for additional relevant information, including any information that may be in the customer's possession. Furthermore, system validation of this product designed by Quectel within a larger electronic system remains the responsibility of the customer or the customer's system integrator. All specifications supplied herein are subject to change.

Copyright

This document contains proprietary technical information which is the property of Quectel Limited. The copying of this document, distribution to others, and communication of the contents thereof, are forbidden without express authority. Offenders are liable to the payment of damages. All rights are reserved in the event of a patent grant or registration of a utility model or design. All specification supplied herein are subject to change without notice at any time.

Copyright © Quectel Wireless Solutions Co., Ltd. 2012

Contents

Contents.....	2
Figure Index	3
0. Revision history	4
1. Introduction	5
1.1. Reference.....	5
1.2. Terms and abbreviations	5
2. FOTA flow	6
2.1. Flow for Programming	6
2.2. Package Tool	7
3. Definition of watchdog.....	9
4. FOTA API functions	10
4.1. Update initialization	10
4.2. Write data to temporary area	10
4.3. Writing data finished	10
4.4. Start update.....	11
5. Example.....	12

Figure Index

FIGURE 1: OPENCPU FOTA FLOW	6
FIGURE 2: FOTA FLOW FOR PROGRAMMING	7
FIGURE 3: OPENCPU FOTA PACKAGE TOOL.....	8

0. Revision history

Revision	Date	Author	Description of change
1.0	2012-09-05	Ablaze LU	Initial

1. Introduction

This document describes the method of remotely updating the embedded application, and how to program and use the package tool.

The solution that this document illustrates applies to the following modules:

- M80 OpenCPU module
- M12 (R2.0) OpenCPU module

1.1. Reference

Table 1: Reference

SN	Document name	Remark
[1]	OPEN_CPU_DGD	OpenCPU Development Guide

1.2. Terms and abbreviations

Table 2: Terms and abbreviations

Abbreviation	Description
API	Application Programming Interface
FOTA	Firmware Over The Air
MD5	Message Digest Algorithm 5

2. FOTA flow

FOTA update process consists of the steps as follows:

- Step 1: compiling codes to generate the new image bin.
- Step 2: packaging the new image bin by 'OpenCPU FOTA Package Tool'.
- Step 3: downloading the new update data, in which the new firmware is included, from the customer server through GPRS, and save data to UFS as a file.
- Step 4: doing updating programmatically with API functions related to FOTA.

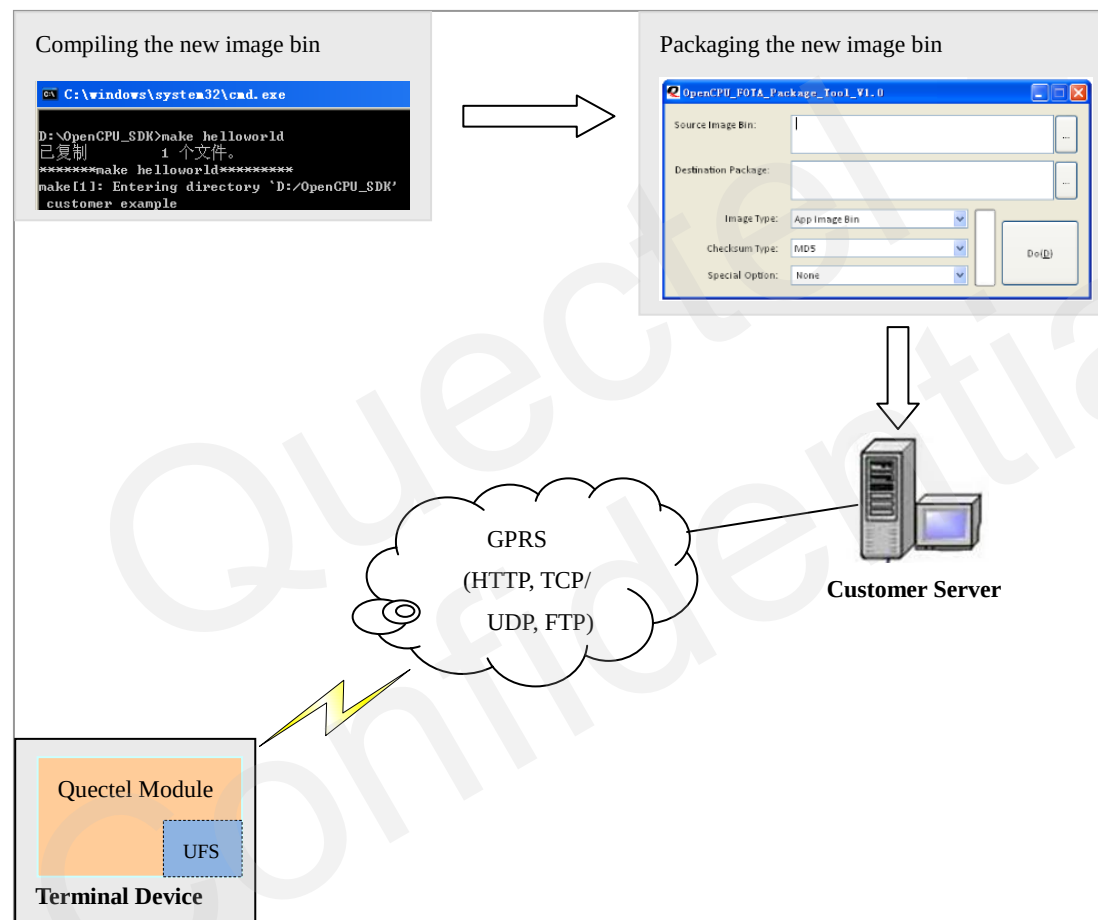


Figure 1: OpenCPU FOTA Flow

2.1. Flow for programming

To implement update locally, the application needs to do the following things:

- ◆ Set watchdog parameters.
- ◆ Call `Ql_Fota_App_Init` function, which initializes parameters that will be used during updating.
- ◆ Call `Ql_Fota_App_Write_Data` function, which reads data from storage and writes them

- to the [cache](#) for updating.
- ◆ Call Ql_Fota_App_Finish function. This function will compare calculated checksum with image checksum in the header after whole image is written and mark the status to UPDATE_NEEDED.
- ◆ Call Ql_Fota_Update function. This function will start FOTA update. Update flag will be written to temporary data area, and then restart module. After that, update is completed.

Totally, the flow chart is as below:

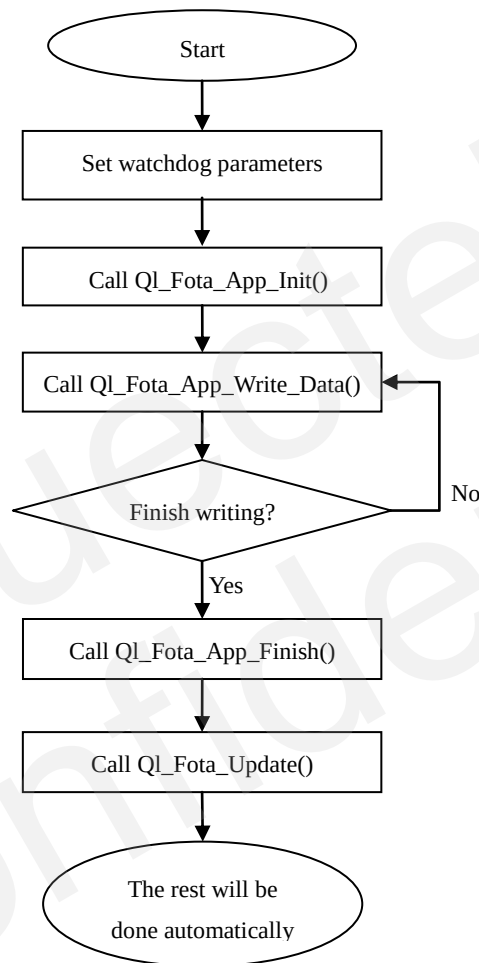


Figure 2: FOTA Flow for Programming

2.2. Package tool

The new firmware needs to be packaged with the tool “Quectel FOTA Package Tool”.

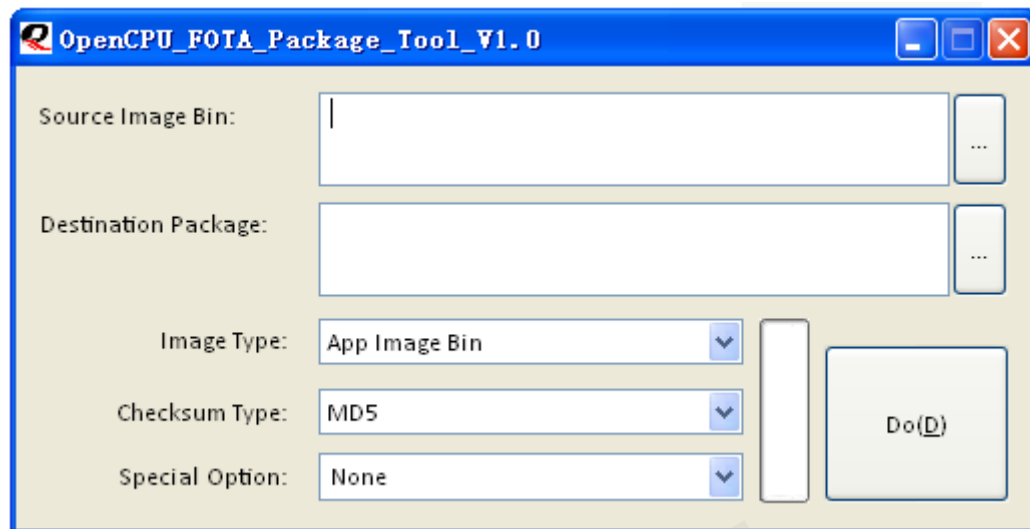


Figure 3: OpenCPU FOTA Package Tool

- Source Bin and Destination Bin

Please select the app bin that you want to package from the “Source Image Bin” column. Select the storage location of the bin that you packaged from “Destination Package”, also you can change the name of the file that you have packaged.

- Image Type

Currently “Image Type” only supports App update, and dose not support Core update. So you can only choose “App Image Bin”.

- Checksum Type

Checksum Type supports “MD5” algorithm. With this choice, checksum data will be written into the destination file. After the update data is downloaded from server and written into module, the module will do checksum to check if there lost any data before the update data reach the module.

- Special Option

This item is for some special purposes. In general, you need to select “None”.

3. Definition of watchdog

OpenCPU FOTA may pull down/up two GPIOs during updating. With this, the external watchdog can be feed during updating (application is not running). Application can specify the GPIO/GPIOs according to the design. Though it is recommended that you add watchdog function in OpenCPU design, you can set *Q_gpio_pin1* and *Q_gpio_pin2* to -1 to disable this function if you do not use external watchdog or do not need feed watchdog during updating.

The watchdog parameters are defined as follows:

```
typedef struct _TFeedDog          // Define Watchdog
{
    s16 Q_gpio_pin1;               // Watchdog GPIO pin 1, If only use one GPIO,
                                // You can set another to -1, which means invalid.
    s16 Q_feed_interval1;         // Time interval of GPIO1 for feeding dog.
    s16 Q_gpio_pin2;               // Watchdog GPIO pin 2, If only use one GPIO,
                                // You can set another to -1, which means invalid.
    s16 Q_feed_interval2;         // Time interval of GPIO2 for feeding dog.
    s16 reserved1;                // Reserve 1
    s16 reserved2;                // Reserve 2
}FEED_DOG;
```

4. FOTA API functions

4.1. Update initialization

Function: s32 Ql_Fota_App_Init(FEED_DOG *pDog);

Description: Initialize parameters that used for updating.

Parameters: Set watchdog parameters.
e.g. Set watchdog pin and the period of feeding dog.

Return: QL_RET_OK indicates the function succeeds.
Negative indicates failure. Please see Error Code Definition.

4.2. Write data to temporary area

Function: s32 Ql_Fota_App_Write_Data(s32 length, s8* buffer);

Description:

FOTA writes data to update cache.

1. This function is used to write data to spare image pool.
2. This API only allows sequentially writing mechanism.
3. Authentication mechanism is executed during writing.

Parameters:

Length: the length of writing (Unit: Bytes).
Buffer: a pointer point to the start address of buffer.

Return: QL_RET_OK indicates the function succeeds.
Negative indicates failure. Please see Error Code Definition.

Note: *The data that is written each time should not be too large. 512 bytes in length are allowed when calling this function and it can be called repeatedly.*

4.3. Writing data finished

Function: s32 Ql_Fota_App_Finish(void);

Description:

FOTA finalizes API.

1. Compare calculated checksum with image checksum in the header after whole image is written.

2. Mark the status to UPDATE_NEEDED.

Parameters: None

Return: QL_RET_OK indicates the function succeeds.
Negative indicates failure. Please see Error Code Definition.

4.4. Start update

Function: s32 Ql_Fota_Update(void);

Description:

Start FOTA update. Update flag will be written to temporary data area, and then restart module. After that, update is finished.

Parameters: None

Return: QL_RET_OK indicates the function succeeds.
Negative indicates failure. Please see Error Code Definition.

5. Example

The parameters of watchdog are configured as follows:

- 1). If you don not need to feed external watchdog, set as follows:

```
FEED_DOG    Q_t_Watch_dog;
Ql_memset((void *)&Q_t_Watch_dog, 0, sizeof(FEED_DOG));
Q_t_Watch_dog.Q_gpio_pin1 = -1;    // If you do not use watchdog you can set to -1,
                                   //-1 means invalid.
Q_t_Watch_dog.Q_feed_interval1 = 200;
Q_t_Watch_dog.Q_gpio_pin2 = -1;
Q_t_Watch_dog.Q_feed_interval2 = 200;
Ql_Fota_App_Init(&Q_t_Watch_dog); // Initialize the update operation
```

- 2). If you need to feed external watchdog, set as follows:

```
FEED_DOG    Q_t_Watch_dog;
Ql_memset((void *)&Q_t_Watch_dog, 0, sizeof(FEED_DOG));
Q_t_Watch_dog.Q_gpio_pin1 = Ql_GetGpioByName(QL_PINNAME_DTR); // Select the pin
                                                             // according to the actual
Q_t_Watch_dog.Q_feed_interval1 = 200;
Q_t_Watch_dog.Q_gpio_pin2 = Ql_GetGpioByName(QL_PINNAME_NETLIGHT);
Q_t_Watch_dog.Q_feed_interval2 = 200;
Ql_Fota_App_Init(&Q_t_Watch_dog); // Initialize the update operation
```

QUECTEL



Shanghai Quectel Wireless Solutions Co., Ltd.

Room 501, Building 13, Tianzhou Road, Shanghai, China 200233

Tel: +86 21 5108 6236

Mail: info@quectel.com